

Binary CLOB 撮合引擎

Binary Central Limit Order Book

系统架构设计文档 · 核心技术白皮书 v4.2

面向二元结果市场 (Binary Outcome Markets) 的高性能中央限价订单簿引擎

核心指标	规格
撮合延迟 (P99)	< 10 μ s (热路径无锁设计, SPSC RingBuf)
订单类型	Limit(GTC/GTD) / Market / IOC / FOK / PostOnly / OrderUpdated
过期精度	1 ms (三层层级时间轮 L0/L1/L2)
Shard 扩展	按 NUMA 节点线性扩展, 每物理核独立 Shard
快照恢复	毫秒级 (zstd level-3 + msgpack rmp_serde)
结算模式	Mint (铸造) / Burn (销毁) / Transfer (转让)
持久化保障	WalWorker 异步写入 + CRC32 校验 + 原子 rename Snapshot
WAL 架构	WalWorker 独立线程, Shard 主循环零磁盘 IO, 批量 fsync
可观测性	OpenTelemetry OTLP Metrics + tracing 结构化日志
背压机制	DealForwarder Ring 70% ON / 40% OFF 迟滞控制
死信队列	DLQ JSONL 追加写, 结算超重试限后持久化
序列号保障	recover_event 同步 next_order_seq, 防重放误拒
OrderBook GC	pop 末尾 POP_GC_THRESHOLD=64 局部触发; 空闲期 maybe_gc() 兜底; ExpiryEngine 增量 GC (batch=64)

基于 Rust 构建 · NUMA 感知调度 · 无锁核心路径 · WalWorker 异步持久化 · 亚微秒级撮合延迟

Author: Jonathan

© Binary CLOB Project — Confidential

1. 项目概述

Binary CLOB（Central Limit Order Book）是一个专为二元结果市场设计的高性能撮合引擎，使用 Rust 语言从零构建。其核心目标是在多事件并发场景下提供亚微秒级撮合延迟、完整的订单生命周期管理，以及生产级别的可靠性保障，包含 WAL 异步写入（WalWorker）、崩溃恢复、背压控制、死信队列等完整工程体系。

1.1 适用场景

- 预测市场（Prediction Markets）：对二元事件（YES/NO）进行交易
- 体育 / 选举 / 金融事件赔率市场
- 链上或链下的二元期权结算系统

1.2 核心业务模型

每个 Event（事件）代表一个二元问题，用户可对该事件下 YES 单或 NO 单。系统在 Message Listener 入口处完成 canonicalize_to_yes() 规范化，将 NO 单转换为 YES 视角，统一在单一订单簿中撮合，极大简化撮合逻辑。

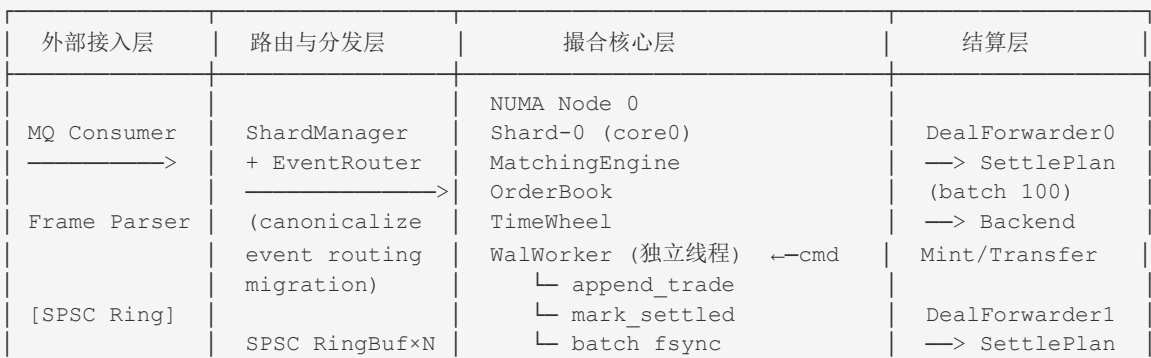
规范化规则	原始订单	规范化结果	入队端
NO Buy @ 40%	买入 NO @ 40%	YES Sell @ 60% (= 100% - 40%)	asks 端
NO Sell @ 60%	卖出 NO @ 60%	YES Buy @ 40% (= 100% - 60%)	bids 端
YES Buy @ X%	买入 YES @ X%	不变	bids 端
YES Sell @ X%	卖出 YES @ X%	不变	asks 端

规范化原理：YES 价格 + NO 价格 = PRICE_BASE（= 1,000,000，即 100%）。规范化后，YES/NO 的互补关系通过 flags 位域（NO bit）保留，用于结算类型（Mint vs Transfer）的判断。

2. 系统架构

2.1 整体架构图

系统分为四层：外部接入层 → 路由与分发层 → 撮合核心层 → 结算层，各层之间通过无锁 SPSC RingBuffer 通信，确保零锁竞争的关键路径。v4 版本新增 WalWorker 独立线程，将磁盘 IO 从 Shard 主循环彻底分离。



Signal	(1M slots each)		
SIGTERM		Shard-1 (core1)	DLQ Writer
SIGINT	Snapshot Chan	MatchingEngine	(JSONL append)
	SnapshotManager	WalWorker (独立线程)	
ClientNotify	WAL Dir		ClientNotifier
[tokio task]	[single thread]	NUMA Node 1..N	[1 thrd/fwd]

2.1.1 服务边界说明

服务边界	组件	线程模型	通信方式
外部接入层	MQ Consumer + Frame Parser + Message Listener	单 tokio task, CPU 亲和性绑定	外部 MQ → Bytes 帧
路由分发层	ShardManager + EventRouter	单线程 (&mut self), 独占调度	SPSC RingBuf → Shard
撮合核心层	Shard + MatchingEngine + TimeWheel	每 Shard 独占 1 物理核	内部同步, 结果推 Forwarder
WAL 持久化层	WalWorker (v4 新增)	每 Shard 1 独立 WAL 线程	WalCmd channel (非阻塞投递)
结算层	DealForwarder + SettlementBackend	每 Forwarder 1 线程	SPSC RingBuf ← Shard
持久化层	WAL / Snapshot / DLQ	写入由 WalWorker/Forwarder 驱动	磁盘 I/O, 异步 fsync
通知层	ClientNotifier (trait)	Forwarder 线程内同步调用	trait 实现 (WebSocket/Kafka)

2.2 外部接口描述

2.2.1 入站帧协议 (TCP / MQ)

系统通过自定义二进制帧协议接收外部订单和命令，帧结构如下：

帧头 (12 字节固定)：

```

magic(4B LE) | version(2B LE) | type(2B LE) | payload_len(4B LE)
magic    = 0xDEADBEEF  - 帧边界校验
version  = 1           - 版本兼容检查
type     = FrameType  枚举：
    1 = OrderCreated  (payload: Order 结构体, 76B fixed, repr(C))
    2 = OrderUpdated  (payload: old_order_id u64 + Order 76B = 84B)
    3 = OrderCanceled (payload: Order 结构体, 76B)
    4 = EventCreated  (payload: event_id u64 LE, 8B)
    5 = EventRemoved  (payload: event_id u64 LE, 8B)

```

2.2.2 系统信号 (Signal)

信号	触发条件	处理行为
SIGTERM	容器编排/运维优雅停机	ShardManager.shutdown(), 广播 Quit 给所有 Shard, 等待 JoinHandle
SIGINT	Ctrl+C 开发调试	同 SIGTERM, 触发 Shard 主循环 break, WAL close + flush

2.3 消息队列与信道

队列/信道	生产者	消费者	类型	容量
Shard 入站 RingBuf	ShardManager（单线程）	Shard 主循环	SPSC lock-free	DEFAULT_RINGBUFF_SIZE = 1M
Forwarder 入站 RingBuf	Shard 主循环（flush_trades）	DealForwarder 线程	SPSC lock-free	DEFAULT_RINGBUFF_SIZE = 1M
WalCmd Channel（v4 新增）	Shard 主循环（非阻塞 try_send）	WalWorker 线程	crossbeam bounded	wal_worker_channel_capacity（默认 4096）
Snapshot Channel	Shard（EventMigrateOut）	ShardManager（poll）	crossbeam unbounded	无界
WAL Done Channel	DealForwarder（结算成功回调）	Shard 主循环（mark_settled）	crossbeam bounded	wal_worker_channel_capacity

2.4 消息与存储交互

2.4.1 用户通知（ClientNotifier）

ClientNotifier 是一个 trait，由 DealForwarder 线程在结算完成后同步调用，生产实现需对接 WebSocket 网关 / Kafka / NATS 等推送通道：

```
pub trait ClientNotifier: Send + Sync + 'static {
    fn notify_order_rejected(&self, order_id: u64, reason: RejectReason);
    fn notify_order_cancelled(&self, order_id: u64);
    fn notify_order_expired(&self, order_id: u64);
    fn notify_trade_created(&self, trade: &Trade);
    fn notify_settlement_failed(&self, buyer_order_id: u64,
                               seller_order_id: u64, event_id: u64);
}
```

2.4.2 WAL（预写日志）

操作	触发时机	格式	fsync 策略
append_trade	flush_trades → WalCmd::Append（v4:非阻塞投递）	magic(4B)+len(4B)+crc32(4B)+rmp_serde(Trade)	WalWorker 批量 sync，每 wal_sync_batch 条 sync_data()
mark_settled	DealForwarder 结算成功回调 → WalCmd::MarkSettled	magic(4B)+len(4B)+crc32(4B)+trade_seq(8B)	✓ need_sync = true（v4.2 已修复）：SETTLED record 写入后触发批量 fsync：mark_settled() 内部 flush() 已移除
replay_wal	EventMigrateIn / 崩溃恢复	按 TRADE/SETTLED magic 分类重放，CRC 校验	只读
delete	Event 正常结束，所有 settlement acked	WalCmd::DrainAndDelete → drain_written_seqs + remove_file	drop file handle 后 remove_file

3.2 SPSC 无锁队列设计

Message Listener → Shard，以及 Shard → DealForwarder，均采用自研的 Single Producer / Single Consumer 无锁 RingBuffer：

特性	实现细节
容量	DEFAULT_RINGBUFF_SIZE = 1 << 20（1M 槽位），必须为 2 的幂，使用位掩码取模
内存布局	Box<[UnsafeCell<MaybeUninit<T>>]>, head/tail 各自 CacheLine<AtomicUsize> 独立 cache line，消除 false sharing
Producer 写入	head.load(Relaxed) + tail.load(Acquire) → ptr::write → head.store(Release)
Consumer 读取	tail.load(Relaxed) + head.load(Acquire) → fence(Acquire) → ptr::read → tail.store(Release)
批量操作	push_batch / pop_batch 支持环绕拷贝（两次 copy_nonoverlapping），单次 head.store
Drop 安全	Drop impl 遍历 tail..head, assume_init_drop() 所有未消费元素，防止内存泄漏

3.3 WalWorker 异步持久化（v4 核心新增）

v4 版本将 WAL write + fsync 从 Shard 主循环完全摘除，消除每批 TradeCreated 的同步 fsync 停顿，显著降低撮合延迟抖动。

3.3.1 WalCmd 命令集

命令	发送时机	是否阻塞发送	说明
Open { event_id }	EventCreated / MigrateIn	阻塞（send_blocking）	打开或创建指定 event 的 WAL 文件
Append { trade }	flush_trades 每条 TradeCreated	非阻塞（try_send）	幂等：同一 trade_seq 在进程内只写一次
MarkSettled { trade_seq }	Shard 收到 DealForwarder ack	非阻塞	写 SETTLED 记录，从 written_seqs 移除
RegisterReplayed { seq }	MigrateIn WAL replay 完成后	阻塞	注册 replay 的 trade_seq，无需写文件
Close	Shutdown 路径	非阻塞	flush + sync，不删文件
CloseSync { ack }	MigrateOut 前	阻塞	同步关闭，Worker 完成后回复 ack
DrainAndDelete	EventRemoved 正常结束	非阻塞	drain written_seqs + delete 文件
Shutdown	Shard 退出	非阻塞 try_send	Worker 收到后退出线程

3.3.2 WalWorker 数据流

```
Shard 主循环
├─ WalCmd::Append(trade)      ← 非阻塞 try_send (channel 满时降级打 WARN)
├─ WalCmd::MarkSettled(seq)   ← 非阻塞
WalWorker 线程（独立，CPU 亲和性绑定）
├─ recv() 阻塞等第一条
├─ try_recv() 批量取后续（最多 wal_sync_batch × 4 条）
├─ append_trade（幂等保护：written_seqs HashSet）
├─ mark_settled (need_sync = true → 下一批次 force_sync 统一落盘，v4.2 已修复)
```

↓ 批量完成后 `force_sync()` 统一 `fsync` 一次
磁盘 (NVMe/SSD)

降级保护: WalCmd channel (容量 4096) 满时 Append 命令投递失败, 打 WARN 日志, `trade_seq` 不写入 WAL。此为降级模式, `trade` 仍然通过 DealForwarder 结算, 最坏情况是崩溃后重放时已结算 `trade` 被重放 (后端需幂等)。

3.4 Shard 主循环延迟策略

阶段	条件	操作	延迟级别
热路径	<code>idle_spin < 200</code>	<code>spin_hint::spin_loop()</code>	纳秒级
中路径	<code>200 ≤ idle_spin < 1000</code>	<code>std::thread::yield_now()</code>	微秒级
冷路径	<code>idle_spin ≥ 1000</code> (重置后)	<code>sleep(50μs)</code>	50μs

3.5 Message Listener 低延迟消费

Message Listener 采用时间驱动 + 空闲让出模式, 消除原始 50ms 固定 tick 导致的订单入站延迟:

```
let mut last_migration_poll = Instant::now();
const MIGRATION_POLL_INTERVAL: Duration = Duration::from_millis(50);

loop {
    // migration check 独立时间驱动, 不阻塞 MQ 消费
    if last_migration_poll.elapsed() >= MIGRATION_POLL_INTERVAL {
        m.poll_pending_migration()?;
        last_migration_poll = Instant::now();
    }

    // MQ 有消息立刻处理 (非阻塞批量消费)
    let mut count = 0;
    while count < MAX_BATCH_PER_TICK {
        match mq.try_recv() {
            Some(bytes) => { dispatch_from_bytes(...); count += 1; }
            None => break,
        }
    }

    // 空闲时让出 tokio worker, 避免独占
    if count == 0 { tokio::task::yield_now().await; }
}
```

4. NUMA 亲和性与 CPU 调度策略

4.1 NUMA 节点绑定

MatchingEngine 与 DealForwarder 必须使用同一 NUMA 节点的 CPU 核心, 以最小化跨节点内存访问 (NUMA remote access penalty 约 2-5×)。ShardManager 在初始化时按 NUMA 拓扑分配核心:

```
// hardware.rs: collect_all_numa_cpus()
优先级:
1. cgroup v2 cpuset (容器环境) → /sys/fs/cgroup/cpuset/cpus.effective
```

2. cgroup v1 cpuset → /sys/fs/cgroup/cpuset/cpuset.effective_cpus
3. NUMA 拓扑枚举 → /sys/devices/system/node/node{N}/cpulist
4. 降级：所有 CPU 作为单一 NUMA 节点

4.2 CPU 分配策略（v4 含 WalWorker 核）

v4 版本 WalWorker 线程在 WalWorker::spawn() 时接受 core 参数并绑核，与 Shard 主循环共核或独占（由配置决定）

4.3 DealForwarder CPU 配置模式

模式	配置场景	CPU 使用效率	适用场景
独享模式	forwarder_count ≥ shard_count，每 Forwarder 独占 1 核	最高（无调度竞争）	高吞吐生产环境
共享模式	多 Shard 轮询同一 Forwarder 核（i % forwarder_cores.len()）	中等（分时共核）	中等负载
混合模式	单核节点，Shard / WalWorker / Forwarder 同核（WalWorker 与 Forwarder 默认同核）	最低（1 核承担全部）	开发 / 测试

5. 业务模块关联与组织模式

5.1 当前：1:1 模式（含 WalWorker）

当前实现中，每个 Shard 绑定一个独立的 DealForwarder 和 WalWorker 实例，形成严格 1:1:1 拓扑：

```
// ShardManager::new() 循环
for (i, shard_core) in shard_cores.iter().enumerate() {
    let forwarder_core = forwarder_cores[i % forwarder_cores.len()];
    let deal_forwarder = DealForwarder::new(forwarder_core, ...);
    let shard = Shard::spawn(shard_core, deal_forwarder, ...);
    // WalWorker 在 Shard 内部 spawn，绑定 shard_core
    // WalWorker::spawn(config, shard_core, wal_dir)
}
```

拓扑：Shard-0 —[SPSC Ring]—> Forwarder-0（独立 SPSC Ring）
 └─[WalCmd]—> WalWorker-0（独立线程）
Shard-1 —> Forwarder-1
 └─> WalWorker-1
Shard-N —> Forwarder-N
 └─> WalWorker-N

- 优点：零竞争，每条成交路径完全独立，无跨 Forwarder/WalWorker 数据竞争
- 缺点：低负载 Event 所在 Shard 的 Forwarder/WalWorker 线程大量空转，线程数随 Shard 线性增长

5.2 规划：M:N 共享 Forwarder 池

NUMA 组内多个 Shard 共享同一组 DealForwarder 实例，降低线程数量和内存开销（P1 优化项）：

目标拓扑（NUMA 组内）：
Shard-0 └─

```

Shard-1 ┌───> ForwarderPool(1~K Forwarder)  K < N
Shard-N └───

```

实现方向：

- ForwarderPool 持有 Arc<Forwarder> 数组
- Shard push 时按 event_id % pool_size 选择 Forwarder
- 保持 SPSC 语义：每个 Forwarder 仍只有 1 producer (pool 内保证)

5.3 Event 生命周期管理

阶段	触发	ShardManager 操作	Shard 内部操作
EventCreated → Active	外部帧 FrameType::EventCreated	选最低负载 Idle Shard，注册路由 Active{shard}	reset_for_new_event(), 初始 化空 OrderBook, WalCmd::Open
Active → MigratingOut	ShardManager 主动发起（负 载均衡/崩溃恢复）	路由切为 Migrating，缓冲 新订单到 pending_orders	force_gc() + snapshot(), WalCmd::CloseSync + ack, clear()→Idle
MigratingOut → Active（目标）	收到 Snapshot (poll_pending_migration)	persist snapshot，发 MigrateIn 给目标 Shard	load_snapshot(), replay_wal(), WalCmd::Open，重放缓冲订 单
Active → Terminating	外部帧 FrameType::EventRemoved	发 EventRemoved 命令，状态 Terminating，等 待确认	clear(), WalCmd::DrainAndDelete, 拒绝剩余订单
Shard 死亡重建	心跳超时 > 5s 或 is_finished()	中止相关迁移， 从 Snapshot 恢 复，同步 next_order_seq	新线程 spawn，加载最新快 照，WalCmd::Open 重建 WalWorker

5.4 FOK 订单 Undo Log 回滚

Fill-Or-Kill 订单通过增量 Undo Log 实现 O(k) 精确回滚（k = 已匹配 maker 数）：

- 撮合前记录每个被消耗的 maker（完整移除或部分消耗前的状态）
- FOK 部分成交时，逆序回放 Undo Log，还原 maker 并放回队列头部
- 成交记录通过 out.truncate() 原子丢弃，无需额外清理
- Undo Log 缓冲区（fok_undo_buf）通过 swap trick 复用，避免每次 FOK 堆分配

6. 存储与内存管理策略

6.1 三级存储架构

层级	存储位置	数据内容	访问频率	触发迁移条件
热数据（L1）	内存	OrderBook (bids/asks)	纳秒级，每消息	Shard 崩溃 / Event 迁移 / 正常 关机

		BTreeMap)、TimeWheel slots、trades Vec、SPSC RingBuf、WalCmd channel		
中间数据（L2）	本地磁盘	WAL 文件、Snapshot 文件（zstd 压缩）、DLQ 文件	毫秒级，WalWorker 异步写	结算成功→WAL delete；Event 结束→Snapshot delete
远端数据（L3）	结算后端（链上/数据库）	最终结算状态、持仓记录、代币余额	秒级，批量提交	SettlementPlan::execute() 成功后由 Backend 写入

6.2 内存占用分析

组件	内存估算	备注
单个 SPSC RingBuf（Shard 入站）	1M × ~100B ≈ 100 MB	IngressMessageRequest，每 Shard 一个入站 Ring
Forwarder RingBuf	1M × ~80B ≈ 80 MB	OutgressMessageRequest，每 Forwarder 一个
WalCmd Channel	4096 × ~80B ≈ 320 KB	bounded channel，每 WalWorker 一个
OrderBook（1000 订单）	~76 KB	Order 结构体 76B（repr(C)固定），BTreeMap 索引
TimeWheel（空载）	~24 KB / Event	L0(1024)+L1(1024)+L2(1024) 槽位，Vec<Entry>
TimeWheel（百万过期单）	~8 MB / Event	entries 满时，canceled HashSet 约额外 8MB
WAL written_seqs	8B × 成交数	HashSet<u64>，Event 结束时全部释放

6.3 数据流转与生命周期

撮合成功 → Trade 数据流（v4 WalWorker 异步路径）：

1. Shard: flush_trades → WalCmd::Append(trade)

[非阻塞投递 WalWorker]

2. WalWorker: append_trade(trade)

[磁盘 L2，先行写入，批量 fsync]

3. Shard: trades.push(TradeCreated(trade))

[内存 L1, trades Vec]

4. flush_trades → Forwarder.push_batch()

[SPSC RingBuf L1]

5. Forwarder: settlements.push(plan)

[内存积攒，batch 100]

6. flush_settlements: backend.execute()

[L3 写入，最多重试 3 次]

6a. 成功: WalCmd::MarkSettled(trade_seq)

[WalWorker 写 SETTLED, flush]

6b. 失败×3: DLQ.push(trade, err)

[L2 JSONL 持久化]

7. Event 结束: WalCmd::DrainAndDelete

[WalWorker drain + delete]

Snapshot 数据流：

迁移 → force_gc()

→ WalCmd::CloseSync{ ack } + 等待 ack

[WalWorker 同步关闭]

→ Shard.snapshot()

→ snapshot_tx.try_send(snap)

[Snapshot Channel]

→ ShardManager.poll() 收到

→ SnapshotManager.save_snapshot()	[L2 原子写 .tmp→rename]
→ MigrateIn: load_snapshot()	[L2→L1 反序列化+解压]
→ WalCmd::Open{event_id}	[WalWorker 打开新 WAL]
→ 迁移完成: delete_snapshot()	[L2 清理]

6.4 内存与存储优化建议

- WAL 批量 sync: WalWorker 每批次结束后统一 force_sync(), NVMe 约 50-200μs/次, 远低于每条独立 fsync
- WalCmd channel 容量: 当前 4096, channel 满时 Append 降级打 WARN, 建议监控 pending_count 指标
- OrderBook GC: 双阈值设计 — pop_best_ask/bid 成功返回前检查 ORDERBOOK_POP_GC_THRESHOLD (=64), 满足即触发 gc_inner(); 空闲期 maybe_gc() 检查全局 ORDERBOOK_GC_THRESHOLD (=1024) 兜底; ExpiryEngine 采用增量 GC (每 tick 最多清理 INCREMENTAL_GC_BATCH=64 条, 触发阈值 INCREMENTAL_GC_TRIGGER=256)
- RingBuf 容量配置化: HARD_MAX_TRADES (32768)、DEFAULT_RINGBUFF_SIZE (1M) 建议从配置文件注入
- Rolling Snapshot: 建议保留最近 N=3 个快照, 防止唯一快照损坏导致数据丢失

7. 订单类型与结算逻辑

7.1 支持的订单类型

类型	标志位	行为描述
Limit GTC	bit1=1, expire_ts=0	挂单, 永不过期, 直至成交或主动取消
Limit GTD	bit1=1, expire_ts>0	挂单, 到期后由时间轮 (1ms 精度) 自动过期
Market	bit1=0	以任意价格立即成交, 剩余量拒绝, 不挂单
IOC	bit2=1	立即成交部分, 剩余量直接丢弃, 不挂单
FOK	bit3=1	必须完全成交, 否则通过 Undo Log 全部回滚
PostOnly	bit7=1	仅挂单, 如会立即成交则拒绝 (防 Taker)
OrderUpdated	—	原子替换: 先取消旧单 (old_order_id), 再提交新单

7.2 结算类型决策

结算类型	判断条件	操作说明
Mint (铸造)	maker 与 taker 互补 (一 YES 一 NO, NO	双方无对应持仓, 铸造新代币

	flags bit 判断)	对, 锁定 qty × COLLATERAL_UNIT 抵押品
Burn (销毁)	maker 与 taker 互补, 且为平仓方向	双方持有互补头寸, 销毁代币对, 释放抵押品
Transfer (转让)	其余情况 (同向方、有价格改善等)	存量代币从卖方转移至买方, collateral = 0

7.3 三层层级时间轮 (HierarchicalTimeWheel)

层级	精度	覆盖范围	槽位数
L0	1 ms	0 ~ 1,024 ms (约 1 秒)	1024
L1	1024 ms	1 ~ 1,048,576 ms (约 17 分钟)	1024
L2	~17 min	17 min ~ 12.4 天	1024

关键设计: 逻辑删除 (canceled HashSet), GC 阈值 8192; L1/L2 cascade 到低层时过滤已取消 entry, 减少 L0 扫描开销; 超出最大范围 (>12 天) 的订单放入 L2 最后一槽兜底, 不静默丢失。

8. 可观测性

8.1 Metrics (OpenTelemetry OTLP)

指标名称	类型	说明

8.2 健康检查

- ShardManager 每秒检查所有 Active Shard 的 last_active_ms
- 超过 SHARD_DEAD_THRESHOLD_MS (5000ms) 无活动 → 疑似死亡, 触发自动恢复
- JoinHandle::is_finished() 检测线程已退出 (panic) → 立即重建, 二次确认防 TOCTOU
- 迁移中的 Shard (source + target) 跳过心跳检查, 避免误判
- flush_trades 阻塞等待时每 1ms 更新 last_active_ms, 防止心跳超时误判
- Shard 重建时同步 next_order_seq = max(next_order_seq, snapshot.last_processed_seq + 1), 防止防重放误拒新订单 (v4 新增)

8.3 结构化日志

- 使用 tracing crate, 支持 span 层级与字段注入
- 关键操作 (EventCreated / Migrated / Removed、Snapshot 保存/加载、Shard 重建) INFO 级别
- 背压 ON/OFF 切换: WARN/INFO 级别, 携带 backlog 数量和百分比
- WalWorker 降级 (channel 满 / fsync 失败): ERROR 级别, 携带 trade_seq
- 热路径 (每条消息处理) 使用 trace! 级别, 生产环境默认过滤

- DLQ 写入：WARN 级别携带完整 JSON，DLQ 自身写入失败为 CRITICAL 级别

本文档严格基于源代码生成，所有技术细节与源码实现一一对应。

Author: Jonathan · © Binary CLOB Project — Confidential

如需要免费测试使用，请与 starlinklab@hotmail.com 联系